



UiT The Arctic University of Norway

Department of Computer Science (IFI)

Mapping performance cliffs in SAMtools sort: Memory, parallelism and temporary file I/O.

Benedicte Jansen and Damian Matulis

Inf-2204, autumn 2026

Table of Contents

1	Introduction	1
1.1	Motivation and goal	1
1.2	Research questions	2
2	Related works	2
3	System	3
4	Early environment and scope	3
5	Early timeline	4
6	Discussion	4
	Works cited	5

1 Introduction

High-throughput sequencing generates large amounts of sequence reads data that has to be processed before it can be used in genomic analyses. The sequence alignment map (SAM) format and its compressed binary representation, binary alignment map (BAM), are commonly used formats to store sequencing reads aligned in a specific order, usually by their chromosomal coordinates on a reference genome. (Li et al., 2009) SAMtools is a widely used suite of programs to process high-throughput sequence alignment data, and samtools sort provides a coordinate-based sorting (Li et al., 2009).

Sorting large BAM-files places a large demand on both processing and storage resources. Since BAM files are compressed and can contain several gigabytes of alignment data, they must be decoded before they can be sorted and then encoded again when the sorted output is written. If the alignment data cannot be processed within the memory limit, samtools sort will create temporary files to the SSD, which later must be merged into the final BAM output. This additional storage can make BAM sorting both computationally and I/O intensive and can limit the performance benefits that are provided by additional threads (Weeks and Luecke, 2017).

In this study, the term performance cliff is used when describing a reproduceable reduction in performance caused by a small change in resource configurations. During BAM sorting, a cliff may occur when reduced memory or increased resources causes an increase in temporary-file activity. Under a fixed memory budget, increasing the thread count will reduce the amount of memory that is available per thread.

This project will focus on investigating how per-thread memory setting and thread count interact in samtools sort. There will be measured execution time, memory consumption and temporary file activity across controlled configurations, and want to identify when performance cliffs occur and determine if increased temporary storage activity can explain the cliff.

1.1 Motivation and goal

Genomic datasets can be huge and require sizable computational resources to be processed. By choosing resource setting poorly the analysis time will increase, storage traffic will increase, and the total computational cost will increase. The goal of this project is to

characterize the interaction between memory, parallelism, and temporary storage during BAM sorting, and to give evidence that can support better resource configurations.

1.2 Research questions

1. How does per-thread memory affect execution time, peak memory consumption, and temporary file activity when the thread count is fixed?
2. How does thread count affect execution time and temporary file activity when the total planned sorting memory budget is held approximately constant?
3. At which per-thread memory and thread count do reproducible performance cliffs occur, and how do they correlate with changes in temporary file activity?

We hypothesize that reducing per-thread memory at a fixed thread will increase temporary file activity. Under a fixed memory budget, we expect temporary-file activity to be stable as the thread count changes, while execution time will initially decrease and then increase with increasing thread count.

2 Related works

Li et al. (2009) introduces the SAM format and the SAMtools software suit, which is widely used for working with high-throughput sequencing data. Danecek et al. (2021) later describes the continued development of SAMtools and its use in genomic data-processing pipelines. Weeks and Luecke (2017) optimized SAMtools sorting for large BAM files using OpenMP task parallelism and memory optimization.

A closely related study by Siebelt (2024) analyzed temporary-file management and the performance of samtools sort 1.19.2. The study showed that the available sorting memory can affect number of temporary files and that low-memory configurations can lead to additional merge operations. The memory scaling experiment was performed on a single sorting thread. Our project extends this by varying thread counts and per-thread memory, including comparisons under a fixed memory budget, using an unmodified samtools sort.

3 System

The system received a fixed unsorted BAM file as input. As you can see in figure 1, SAMtools reads and decompresses the BAM records before placing them in sorting buffers in memory. The `-m` parameter controls approximately how much memory is available per sorting-thread, while the `-@` parameter controls the number of sorting and compressing

threads. The configured memory for storing BAM is approximately the multiplication of these two parameters. If the memory limit is hit before the end of the input file, then SAMtools will create temporary BAM files, which are merged and compressed again into after sorting. The performance of this therefore depends on the CPU processing, available memory, thread coordination and storage I/O. Our benchmark will be measuring the time, memory consumption, and temporary file activity for each configuration it tests.

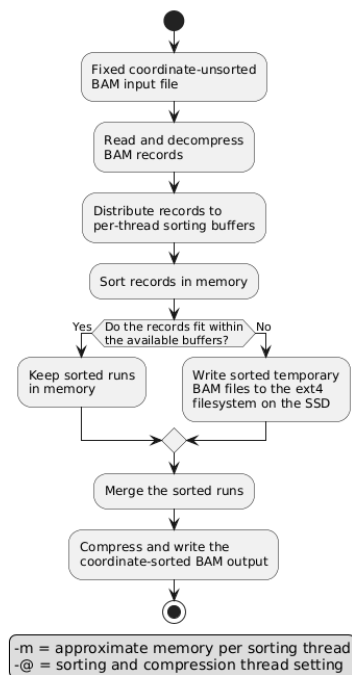


Figure 1: System model for sorting with samtools sort. Made using planttext.com.

4 Early environment and scope

The primary experiment will be using a BAM-file based on data from a study published by Ross-Inness et al., (2012). These files are already coordinate-sorted, and we will generate a non coordinate-sorted versions using samtools sort -n. This file will be generated once and then reused as fixed input thought the experiments.

A second BAM-file will be used as a smaller validation experiment. Selected configurations from before, around and after the found performance cliff will be repeated with this file. This will allow us to observe if the identified performance pattern is specific to the primary BAM file.

When it comes to software and hardware, we are going to test everything on one controlled computer system. The experiments will be run on WSL version 2.7.12.0 with Ubuntu 24.04.2

LTS Windows 11. The system has an AMD Ryzen 9 5900HX with Radeon Graphics, with 8 cores and 16 logical processors, 15.4 GB host RAM, a 7.5 GiB WSL2 memory limit, and an INTEL SSDPEKNU010TZ containing the ext4 experimental filesystem. We will use SAMtools 1.19.2 and HTSlib 1.19.

5 Early timeline

We already have a BAM-file and SAMtools installed, and a good idea of how to approach the next step. The next step will be to create a Benchmark that will measure the performance of the sorting and then conduct the main experiments. We will then repeat selected configurations using the other dataset, analyze the results and prepare for the final report and presentation.

6 Discussion

This project comes with several limitations that can affect our results. Firstly, all primary experiments will be executed on one hardware and software environment. The exact execution times and the location or performance cliff may be different on other CPUs, memory capacities, storage devices, operating systems, and SAMtools versions. This limits the generalizability of our results, but using one fixed system also gives a better controlled environment for comparing the tested memory and thread configurations.

Secondly, the primary experiment will use one BAM-file. BAM files with different sizes or number of alignment records might have different sorting behavior. The project therefore does not aim to find one universally SAMtools configuration. Instead, it aims to characterize the interaction between per-thread memory, thread count, and temporary file activity for a controlled workload. To examine if this pattern is specific to one BAM file, we will also test these configurations using a second BAM file.

Finally, measurements can also be affected by variations caused by background processes, filesystem caching, etc. To reduce this risk, we use the same software and system configurations, unnecessary programs will be closed, and we will repeat each config several times.

Works cited

- Herzeel, C., Costanza, P., Decap, D., Fostier, J., Reumers, J. (2015). elPrep: High-Performance Preparation of Sequence Alignment/Map Files for Variant Calling. *PLOS ONE* 10(7): e0132868. <https://doi.org/10.1371/journal.pone.0132868>
- Li, H., Handsaker, B., Wysoker, A., Fennell, T., Ruan, J., Homer, N., Marth, G., Abecasis, G., Durbin, R. (2009). 1000 Genome Project Data Processing Subgroup, The Sequence Alignment/Map format and SAMtools, *Bioinformatics*, Volume 25, Issue 16, Pages 2078–2079, <https://doi.org/10.1093/bioinformatics/btp352>
- Tarasov, A., Vilella, A. J., Cuppen E., Nijman, E. J., Prins, P. (2015). Sambamba: fast processing of NGS alignment formats, *Bioinformatics*, Volume 31, Issue 12, Pages 2032–2034, <https://doi.org/10.1093/bioinformatics/btv098>
- Weeks, N.T., Luecke, G.R. (2017). Optimization of SAMtools sorting using OpenMP tasks. *Cluster Comput* 20, page 1869–1880. <https://doi.org/10.1007/s10586-017-0874-8>
- Li, H. (2026, July 9). *Samtools sort – sorts SAM/BAM/CRAM files*. Samtools. <https://www.htslib.org/doc/samtools-sort.html>
- Samtools. (n.d). *Samtools*. <https://www.htslib.org>
- Danecek, P., Bonfield, J. K., Liddle, J., Marshall, J., Ohan, V., Pollard, M. O., Whitwham, A., Keane, T., McCarthy, S. A., Davies, R. M., & Li, H. (2021). Twelve years of SAMtools and BCFtools. *GigaScience*, 10(2), giab008. <https://doi.org/10.1093/gigascience/giab008>
- Les kommentar fra ATstar: <https://www.biostars.org/p/9550294/>
- Ross-Innes, C. S., Stark, R., Teschendorff, A. E., Holmes, K. A., Ali, H. R., Dunning, M. J., Brown, G. D., Gojis, O., Ellis, I. O., Green, A. R., Ali, S., Chin, S. F., Palmieri, C., Caldas, C., & Carroll, J. S. (2012). Differential oestrogen receptor binding is associated with clinical outcome in breast cancer. *Nature*, 481(7381), 389–393. <https://doi.org/10.1038/nature10730>
- Siebelt, D. (2024). *Faster Sorting of Aligned DNA-Read Files* [Bachelor’s thesis]. At the KIT Department of Informatics Institute of Theoretical Informatics (ITI).

Vijaykumar, N., Hsieh, K., Pekhimenko, G., Khan, S., Shrestha, A., Ghose, S., Jog, A., Gibbons, P., Mutulu O. (2016). Zorua: A Holistic Approach to Resource Virtualization in GPUs. *In The 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-49)*. IEEE Press, Article 15, 1–14. <https://dl.acm.org/doi/10.5555/3195638.3195656>